

EPAM Senior Software Data Engineer

Round 2 Technical Interview — Questions & Detailed Answers

Focus: SQL • Python • Spark/PySpark • Databricks • Delta Lake • AWS • Data Engineering System Design • Production Scenarios

Interview preparation guide with practical explanations, code examples, senior-level follow-up questions, and production-oriented answers.

Contents

1. SQL & Database
2. Python
3. Spark & PySpark
4. Databricks & Delta Lake
5. AWS
6. Data Engineering Scenarios
7. Production, Monitoring & Troubleshooting
8. End-to-End Data Pipeline Design
9. Rapid-Fire Senior-Level Questions

How to use this guide: First answer each question yourself, then compare with the detailed answer. For senior interviews, focus on the reasoning behind each solution, trade-offs, failure handling, and performance implications.

1. SQL & Database

1.1 Nth Highest Salary

Question: How would you find the Nth highest salary?

Detailed Answer: A window function is usually the clearest approach. **DENSE_RANK()** is useful when duplicate salaries should share the same rank and you want the Nth distinct salary.

```
SELECT *
FROM (
  SELECT e.*,
  DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employee e
) t
WHERE rnk = 3;
```

Senior follow-up: What changes if the interviewer wants the Nth row rather than the Nth distinct salary? Use **ROW_NUMBER()**.

1.2 Rank vs Dense Rank vs Row Number

Question: What is the difference between **RANK()**, **DENSE_RANK()**, and **ROW_NUMBER()**?

Detailed Answer: **ROW_NUMBER()** gives every row a unique sequential number. **RANK()** gives tied rows the same rank and leaves gaps after ties. **DENSE_RANK()** also gives ties the same rank but does not leave gaps.

```
-- salaries: 100, 100, 90, 80
ROW_NUMBER(): 1, 2, 3, 4
RANK(): 1, 1, 3, 4
DENSE_RANK(): 1, 1, 2, 3
```

Senior follow-up: Ask yourself which function is correct for 'top 3 salaries' versus 'top 3 employees'.

1.3 Employees Above Department Average

Question: Find employees whose salary is greater than the average salary of their own department.

Detailed Answer: The subquery must calculate one average per department and then match each employee to the average for that department. A window function is concise and interview-friendly.

```
SELECT *
FROM (
  SELECT e.*,
  AVG(salary) OVER (PARTITION BY department) AS avg_salary
  FROM employee e
) t
WHERE salary > avg_salary;
```

```
-- Alternative using JOIN
SELECT e.*
FROM employee e
JOIN (
  SELECT department, AVG(salary) AS avg_salary
  FROM employee
  GROUP BY department
) d
ON e.department = d.department
WHERE e.salary > d.avg_salary;
```

Senior follow-up: The common mistake is comparing salary with a subquery that returns multiple department averages.

1.4 Running Total

Question: Calculate a running total of sales for each customer.

Detailed Answer: Use SUM() as a window function with an ordering column. Partitioning resets the running total for each customer.

```
SELECT customer_id,  
order_date,  
amount,  
SUM(amount) OVER (  
PARTITION BY customer_id  
ORDER BY order_date  
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
) AS running_total  
FROM orders;
```

1.5 LAG and LEAD

Question: What are LAG() and LEAD() used for?

Detailed Answer: LAG accesses a previous row and LEAD accesses a following row without requiring a self-join. They are useful for comparing current versus previous transactions, detecting changes, calculating intervals, and identifying trends.

```
SELECT customer_id,  
order_date,  
amount,  
LAG(amount) OVER (  
PARTITION BY customer_id ORDER BY order_date  
) AS previous_amount,  
LEAD(amount) OVER (  
PARTITION BY customer_id ORDER BY order_date  
) AS next_amount  
FROM orders;
```

1.6 WHERE vs HAVING

Question: What is the difference between WHERE and HAVING?

Detailed Answer: WHERE filters rows before grouping/aggregation. HAVING filters groups after GROUP BY and aggregation. A common optimization is to filter non-aggregated rows as early as possible with WHERE when logically possible.

```
SELECT department, AVG(salary) AS avg_salary  
FROM employee  
WHERE salary IS NOT NULL  
GROUP BY department  
HAVING AVG(salary) > 70000;
```

1.7 UNION vs UNION ALL

Question: What is the difference between UNION and UNION ALL?

Detailed Answer: UNION combines result sets and removes duplicate rows, which generally requires additional work. UNION ALL simply appends results and preserves duplicates, so it is usually faster when duplicate removal is not required.

1.8 Duplicate Records

Question: How would you identify duplicate records using a business key?

Detailed Answer: Group by the business key and use HAVING COUNT(*) > 1. For row-level duplicate identification, use ROW_NUMBER() and keep the first row according to a deterministic ordering.

```
SELECT customer_id, order_id, COUNT(*) AS cnt  
FROM orders  
GROUP BY customer_id, order_id  
HAVING COUNT(*) > 1;
```

```
SELECT *
FROM (
  SELECT o.*,
  ROW_NUMBER() OVER (
    PARTITION BY customer_id, order_id
    ORDER BY ingestion_time DESC
  ) AS rn
  FROM orders o
) t
WHERE rn = 1;
```

1.9 Slow SQL Query

Question: A query on a multi-billion-row table became much slower. How would you troubleshoot it?

Detailed Answer: Start with the execution plan and identify full scans, expensive joins, large sorts, poor cardinality estimates, spills, and excessive data movement. Then verify statistics, predicates, join conditions, indexes where relevant to the database, partition pruning where supported, and whether the query is processing unnecessary columns or rows. Measure each change rather than guessing.

Senior follow-up: Explain the same investigation differently for an OLTP database versus a Spark/Databricks workload.

1.10 Normal Forms

Question: Explain 1NF, 2NF, and 3NF.

Detailed Answer: **1NF:** atomic values and no repeating groups. **2NF:** in 1NF and every non-key attribute depends on the whole candidate key, especially relevant for composite keys. **3NF:** in 2NF and non-key attributes should not depend transitively on another non-key attribute. In analytics, denormalization is often deliberately used for performance and usability.

2. Python

2.1 List vs Tuple vs Set vs Dictionary

Question: When would you use list, tuple, set, and dictionary?

Detailed Answer: A list is ordered and mutable; a tuple is ordered and immutable; a set stores unique values and is optimized for membership-style operations; a dictionary maps keys to values and is useful for fast key-based lookup. Choose based on semantics rather than only performance.

2.2 Generator

Question: How would you process a 20 GB file without loading it completely into memory?

Detailed Answer: Use streaming or generators so records are processed incrementally. For structured data, use chunked reads or a distributed engine when the workload is large enough. The key idea is bounded memory usage.

```
def read_lines(path):
    with open(path, "r") as f:
        for line in f:
            yield line.rstrip()

for line in read_lines("large_file.txt"):
    process(line)
```

2.3 Shallow vs Deep Copy

Question: What is the difference between shallow copy and deep copy?

Detailed Answer: A shallow copy creates a new outer object but nested mutable objects can still be shared. A deep copy recursively copies nested objects. This matters when modifying nested lists/dictionaries and avoiding unintended side effects.

2.4 Exception Handling

Question: How should exception handling be implemented in a production data pipeline?

Detailed Answer: Catch specific exceptions rather than using a broad except without handling. Log useful context such as job/run ID and input partition, clean up resources where necessary, retry only transient failures, and fail fast for non-recoverable data or configuration problems. Do not silently swallow exceptions.

2.5 Coding: Two Sum

Question: How would you solve Two Sum efficiently?

Detailed Answer: Use a dictionary to store values already seen and their indices. For each value x, check whether target-x has already been seen. This gives average $O(n)$ time and $O(n)$ additional space.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
    return []
```

3. Spark & PySpark

3.1 Spark Architecture

Question: Explain Spark architecture.

Detailed Answer: A Spark application has a driver that creates the Spark session/context and coordinates execution. Executors run tasks and store/cache data. The cluster manager allocates resources. Spark builds a logical execution plan from transformations, optimizes it, and breaks work into stages and tasks, with shuffle boundaries usually separating stages.

3.2 Transformations vs Actions

Question: What is the difference between transformations and actions?

Detailed Answer: Transformations such as filter, select, join, and groupBy describe a new DataFrame/RDD and are lazily evaluated. Actions such as count, collect, show, and write trigger execution.

```
df = spark.read.parquet(path)
df = df.filter("salary > 50000")
df.count() # Action: execution occurs
df = df.groupBy("department").count()
df.show() # Another action
```

Senior follow-up: Yes, an action can appear between transformations. It triggers execution for the lineage required at that point; later transformations form additional work when another action is called.

3.3 Narrow vs Wide Transformation

Question: What is the difference between narrow and wide transformations?

Detailed Answer: A narrow transformation can compute an output partition from a small number of input partitions without redistributing data, such as filter. A wide transformation requires data movement across partitions, such as groupByKey or many joins, and normally causes a shuffle.

3.4 What is Shuffle?

Question: Why is shuffle expensive?

Detailed Answer: Shuffle redistributes records between executors/partitions, involving serialization, network transfer, disk I/O, sorting or hashing, and additional task coordination. Large shuffles can become the dominant cost in a Spark job.

3.5 Broadcast Join

Question: You have a 1 TB fact table and a 10 MB dimension table. How can you optimize the join?

Detailed Answer: Broadcast the small dimension table so each executor can use a local copy and the large table does not need a full shuffle for the join.

```
from pyspark.sql.functions import broadcast

result = fact_df.join(
    broadcast(dim_df),
    "customer_id"
)
```

Senior follow-up: If the dimension becomes several GB, broadcasting may cause memory pressure. Then evaluate shuffle join, partitioning, AQE, and other strategies.

3.6 repartition vs coalesce

Question: What is the difference between repartition() and coalesce()?

Detailed Answer: repartition can increase or decrease partitions and generally causes a shuffle. coalesce is mainly used to reduce partitions with less movement and is useful before writing when reducing file count. The trade-off is that coalesce can create uneven partitions in some cases.

3.7 Data Skew

Question: One customer has hundreds of millions of transactions and the Spark job is extremely slow. What is happening?

Detailed Answer: The key is highly skewed, so one or a few partitions receive disproportionately large amounts of data. Those tasks become stragglers while the other tasks finish earlier. Solutions include salting, AQE skew handling, better partitioning, pre-aggregation, or broadcasting an appropriate side of a join.

3.8 Caching

Question: When should you cache a DataFrame?

Detailed Answer: Cache when the same expensive intermediate dataset is reused by multiple actions or branches and the cost of recomputation is significant. Do not cache everything; caching consumes executor memory and can cause eviction or pressure. Use persist with an appropriate storage level when needed.

3.9 Slow Spark Job

Question: A Spark job that normally takes 10 minutes now takes 60 minutes. How do you troubleshoot it?

Detailed Answer: Start with Spark UI: identify the slow stage, task duration distribution, input/output sizes, shuffle read/write, spills, skewed partitions, and executor failures. Then inspect the physical plan and data changes. Typical causes include skew, a changed join strategy, excessive shuffle, small files, too many partitions, insufficient resources, or a bad filter/join.

3.10 Lazy Evaluation

Question: Why does Spark use lazy evaluation?

Detailed Answer: It allows Spark to build and optimize a complete execution plan before running it. This enables predicate pushdown, column pruning, operator reordering, and other optimizations. Execution starts only when an action requires a result.

4. Databricks & Delta Lake

4.1 Why Delta Lake?

Question: What advantages does Delta Lake provide over plain Parquet files?

Detailed Answer: Delta adds a transaction log and table-level features such as ACID transactions, schema enforcement, time travel, reliable updates/deletes, MERGE, and better management of concurrent writes. Parquet remains the underlying file format, while Delta adds transactional metadata and behavior.

4.2 Incremental RDBMS to Databricks

Question: An RDBMS sends new and updated records every day. How would you implement an incremental load?

Detailed Answer: Prefer CDC when available. Otherwise use a reliable last-modified timestamp or monotonically increasing key with a control/watermark table. Extract only the changed range, land it in Bronze, validate and deduplicate, then apply changes to Silver using MERGE. Store the processed watermark only after successful completion. Design the process so reprocessing the same batch does not create duplicates.

```
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

4.3 Idempotency

Question: What does idempotency mean in a data pipeline?

Detailed Answer: Running the same logical batch more than once should produce the same final state as running it once. This can be achieved with stable business keys, deterministic deduplication, MERGE/upsert semantics, batch identifiers, checkpoints, and careful watermark management.

4.4 Schema Evolution

Question: How would you handle a source adding a new column?

Detailed Answer: First decide whether the change is backward-compatible. Use controlled schema evolution where appropriate, validate incoming schemas, alert on breaking changes, and version contracts when necessary. Do not allow uncontrolled source changes to silently alter critical downstream datasets.

4.5 OPTIMIZE and VACUUM

Question: What are OPTIMIZE and VACUUM used for in Delta Lake?

Detailed Answer: OPTIMIZE compacts files to improve read efficiency and can reorganize data according to supported optimization techniques. VACUUM removes old data files no longer needed under the retention rules. VACUUM should be used carefully because aggressive retention can interfere with time travel or long-running readers.

4.6 Medallion Architecture

Question: Explain Bronze, Silver, and Gold layers.

Detailed Answer: **Bronze:** raw/landed data with minimal transformation and traceability. **Silver:** cleaned, validated, standardized, deduplicated business-level data. **Gold:** business-ready aggregates or dimensional datasets optimized for consumption. The exact implementation varies, but the separation helps reliability, reuse, and governance.

5. AWS

5.1 S3 Data Lake

Question: How would you organize data in S3 for analytics?

Detailed Answer: Use a clear landing/raw/curated structure, consistent naming, appropriate partitioning based on common query predicates, and columnar formats such as Parquet for analytical workloads. Avoid excessive partition cardinality and tiny files. Apply IAM, encryption, lifecycle policies, and monitoring.

5.2 Glue

Question: What is AWS Glue used for?

Detailed Answer: Glue provides serverless data integration capabilities, including ETL jobs, the Data Catalog, crawlers, workflows, and integrations with AWS data stores. It can run Spark-based transformations without managing the underlying cluster infrastructure yourself.

5.3 On-Prem Oracle to AWS

Question: Design a pipeline to move data from an on-premise Oracle database to AWS.

Detailed Answer: A robust design could use an ingestion/CDC mechanism to extract changes, land them in S3, process them with Glue or Databricks/Spark, maintain Bronze/Silver/Gold datasets, and expose curated data to downstream analytics. Add a control table, data-quality checks, monitoring, retries, encryption, IAM, and alerting.

6. Data Engineering Scenarios

6.1 Duplicate Data After Retry

Question: A pipeline failed and was restarted. The second run inserted duplicates. How do you fix the design?

Detailed Answer: Make the pipeline idempotent. Define the correct business key, deduplicate incoming data, use MERGE/upsert semantics where appropriate, track batch/run identifiers, and commit/update watermarks only after successful processing. A retry should safely reproduce the same final state.

6.2 Late-Arriving Data

Question: How would you handle records arriving days after their event date?

Detailed Answer: Separate event time from ingestion time. Use a watermark with an appropriate lateness allowance, support corrections to affected partitions/aggregates, and ensure downstream calculations can be recomputed or incrementally corrected. The right approach depends on whether the business requires exact historical restatement.

6.3 Bad Records

Question: What should happen when 1% of incoming records fail validation?

Detailed Answer: Do not silently discard them. Route invalid records to a quarantine/error dataset with the original payload or sufficient diagnostic information, reason codes, batch ID, and timestamps. Decide whether the business permits partial success or requires the entire batch to fail. Monitor error rates and alert on thresholds.

6.4 Failed Downstream System

Question: Your downstream database is unavailable while the pipeline is processing. What would you do?

Detailed Answer: Decouple ingestion from delivery when possible. Persist the successfully processed dataset in durable storage, retry transient downstream failures with backoff, maintain delivery state, and resume from the last successful checkpoint. Avoid reprocessing the entire upstream pipeline unnecessarily.

7. Production, Monitoring & Troubleshooting

7.1 Production Incident

Question: A critical data pipeline failed at 2 AM. What is your approach?

Detailed Answer: Follow a structured incident process: detect and assess impact, inspect logs/metrics and the failing stage, identify whether the issue is code/data/infrastructure/dependency related, recover safely, validate the output, communicate status, and then perform root-cause analysis. After recovery, add preventive controls such as better validation, alerts, tests, or retry behavior.

7.2 Monitoring

Question: What should you monitor for a production data pipeline?

Detailed Answer: At minimum: job success/failure, duration, freshness/SLAs, input/output row counts, rejected records, schema changes, data-quality metrics, Spark resource usage, shuffle/spill indicators, retries, and downstream delivery status. Monitoring should produce actionable alerts rather than just dashboards.

7.3 Data Quality

Question: How would you implement data validation?

Detailed Answer: Use checks such as null constraints, uniqueness, referential integrity where relevant, accepted-value rules, row-count thresholds, freshness, distribution/anomaly checks, duplicate detection, and reconciliation against source totals. Persist validation results so they are auditable.

7.4 Pipeline Cost Optimization

Question: How would you reduce the cost of a Spark/Databricks pipeline?

Detailed Answer: Process incrementally instead of repeatedly scanning full history, reduce unnecessary columns and rows early, use efficient file formats, control file sizes, optimize joins and shuffles, avoid excessive caching, choose appropriate cluster sizes, use autoscaling/appropriate compute policies, and schedule workloads efficiently. Always balance cost against SLA and reliability.

8. End-to-End Data Pipeline Design

8.1 Design a Customer Transactions Pipeline

Question: Design a scalable pipeline that ingests transactions from an RDBMS, processes changes daily, and serves analytics.

Detailed Answer: A strong architecture separates ingestion, raw storage, transformation, serving, and operational control. Use CDC or a reliable watermark from the RDBMS; land immutable change data in Bronze; validate, deduplicate, and apply business rules in Silver; create customer-level aggregates in Gold; and expose Gold to BI/analytics. Add control metadata, data-quality checks, observability, security, retry/replay capability, and idempotent processing.

```
RDBMS
|
| CDC / Incremental Extract
v
Object Storage / Landing
|
v
Bronze Delta
|
| Clean + Deduplicate + MERGE
v
Silver Delta
|
| Business Aggregations
v
Gold Delta
|
v
BI / Analytics
```

Senior follow-up: Be prepared to justify every layer and explain what happens if any stage fails.

8.2 1 TB Daily Dataset

Question: You receive 1 TB of data every day. How would you process it efficiently?

Detailed Answer: Use distributed processing rather than a single-machine Python process. Read only required columns, filter early, use Parquet/Delta, partition based on access patterns, choose an appropriate number of partitions, optimize joins, avoid unnecessary shuffles, handle skew, and write reasonably sized files. If only a small portion changes, use incremental processing rather than rereading all history.

9. Rapid-Fire Senior-Level Questions

9.1 What is AQE?

Adaptive Query Execution dynamically adjusts parts of the physical execution plan at runtime based on observed statistics, such as handling skew and coalescing shuffle partitions in supported scenarios.

9.2 What is predicate pushdown?

Applying filters as close to the data source as possible so fewer rows are read and processed.

9.3 What is column pruning?

Reading only the columns needed by the query instead of scanning every column.

9.4 Why is Parquet preferred for analytics?

It is columnar, compressed, supports efficient selective reads, and works well with predicate and column pruning.

9.5 What is the small-file problem?

Many tiny files increase metadata overhead and task scheduling and can degrade read/write performance; compaction and better ingestion design help.

9.6 What is a checkpoint?

A durable progress marker used to resume processing safely without starting from the beginning, depending on the framework and pipeline design.

9.7 What is CDC?

Change Data Capture identifies inserts, updates, and deletes from a source so downstream systems can process changes incrementally.

9.8 What is time travel in Delta?

The ability to query historical table versions or restore/recover data using transaction history and retained files, subject to retention policies.

9.9 Why use MERGE?

MERGE supports conditional insert/update/delete behavior based on a matching key, making it useful for upserts and CDC application.

9.10 What makes a pipeline production-grade?

Correctness, idempotency, fault tolerance, observability, data quality, security, scalability, maintainability, controlled deployments, and clear recovery/replay procedures.

Final interview strategy

For EPAM Round 2, avoid giving only definitions. Structure answers as: **concept** → **why it matters** → **example** → **trade-off** → **production consideration**. If you do not know an exact answer, explain how you would investigate it rather than guessing.

High-priority revision: Spark execution and optimization, SQL window functions, incremental/CDC pipelines, Delta MERGE and idempotency, data skew, joins, Databricks architecture, AWS data-lake design, and production

troubleshooting.